

- La clase anterior presentamos una introducción al curso, ilustrando la relevancia de algunos de los temas que estudiaremos.
- Hoy entraremos en materia:
- Revisión del problema general y notación.
- Breve repaso de conceptos de optimización y algoritmos.
- Revisaremos los métodos de Gradiente y Newton
- Despues, hay ayudantía/taller sobre contenidos de matemáticas

# El problema general de Optimización:

- El problema:

$$P) \quad \min_{x \in S} f(x)$$

donde  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  y  $S \subset \mathbb{R}^n$ .

- Muchas veces haremos  $S$  explícito a través de restricciones algebraicas:

$$P) \quad \begin{array}{ll} \min & f(x) \\ \text{s.a.} & g_i(x) \leq b_i, i = 1, \dots, m \end{array}$$

donde  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  y  $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$ .

## El problema general: queremos calcular una solución

- ¿Qué posibilidades tenemos, en general, de poder encontrar una solución a  $P$ )?
- Encontremos el mínimo de  $f(x) = x^2 + e^{-x}$ .
- Veamos que nos dice un chat de IA
- ¿Y si más bien tratamos con una solución **“aproximada”**?
- Por ejemplo,  $\bar{x} = 0,351732281$  es tal que  $f'(\bar{x}) = -3,86 \times 10^{-6}$ .

## El problema general: definiciones básicas

- Sea  $x^*$  solución óptima de  $P$ ).
- Sea  $\epsilon > 0$ .
- Decimos que  $\bar{x} \in S$  es una solución  $\epsilon$ -**óptima** de  $P$ ) si

$$|f(\bar{x}) - f(x^*)| \leq \epsilon$$

- Decimos que  $\bar{x} \in S$  es una  $\epsilon$ -aproximación a la solución óptima de  $P$ ) (o solución  $\epsilon$ -**aproximada**) si

$$\|\bar{x} - x^*\| \leq \epsilon$$

- ( $\|\cdot\|$  denota una norma en el espacio euclideo  $\mathbb{R}^n$ )
- Pregunta: ¿son equivalentes las dos nociones?

# Cómo podemos calcular

- Para resolver el problema usaremos un “algoritmo” que genera una sucesión de puntos, partiendo de  $x^0$ :

$$x^0, x^1, x^2, \dots$$

- El algoritmo usa las iteraciones anteriores y una subrutina a la que se le “preguntan” características de  $f$ .
- Esa subrutina es lo que se suele llamar un “oráculo”.

# Cómo podemos calcular



(Fuente: [historia.nationalgeographic.com.es](http://historia.nationalgeographic.com.es))

# Cómo podemos calcular

- El oráculo puede darnos distintos niveles de información.
  - ▶ **Oráculo de orden cero:** valores de la función.
  - ▶ **Oráculo de primer orden:** valores y derivadas (o algo parecido).
  - ▶ **Oráculo de segundo orden:** valores, derivadas y segundas derivadas.
- Una de las preguntas fundamentales en el curso: ¿Qué tan **eficiente** es el algoritmo?:
- Dado un error aceptable  $\epsilon > 0$ , ¿cuánto tiempo necesitamos para obtener una solución que cumpla con ese error (en alguna medida de aproximación)?
- Analizar esto es determinar la “complejidad del algoritmo”.

# Qué podemos calcular

- Supongamos primero que nuestro algoritmo usa un oráculo de orden cero (sólo valores de la función).
- Tomemos, para simplificar, una función en una variable, y supongámosla continua.
- Veamos si podemos encontrar una solución aproximada para este problema:

$$P) \quad \min_{a \leq x \leq b} f(x)$$

# Qué podemos calcular

- Sigamos con un oráculo de orden cero.
- Supongamos ahora que ponemos limites a la variación de  $f$ .

## Definición

Decimos que  $f : \mathbb{R} \rightarrow \mathbb{R}$  *satisface una **condición de Lipschitz**, con constante  $L$* , si

$$|f(x) - f(y)| \leq L|x - y|, \quad \forall x, y$$

- Suponiendo ahora la hipótesis de Lipschitz, ¿Qué podemos hacer ahora con el problema:

$$P) \quad \min_{a \leq x \leq b} f(x) \quad ?$$

# Qué podemos calcular

- Solo con un oráculo de orden cero no vamos a llegar muy lejos...
- Por esa razón tendremos que ir pidiendo más propiedades a la función y/o al conjunto de restricciones:
  - ▶ Usar oráculos de mayor orden
  - ▶ Pedir estructura más fuerte, como por ejemplo, convexidad.
- Y esas propiedades adicionales permitirán mejorar la eficiencia.

- Vamos a revisar ahora los algoritmos más básicos (y famosos) para un problema no lineal sin restricciones:
- Método del Gradiente y el Método de Newton
- Después avanzaremos más allá.

- El problema a estudiar:

$$P) \quad \min_{x \in \mathbb{R}^n} f(x)$$

donde  $f$  es, al menos, continuamente diferenciable.

# La idea del Algoritmo de Descenso

- Muchos algoritmos se basan en la idea de “descender” hacia el mínimo...

## Definición

$d \neq 0$  se dice una **dirección de descenso** de  $f$  en  $x$  si existe  $r > 0$  tal que  $f(x + \lambda d) < f(x), \forall \lambda \geq 0, \lambda \leq r$ .

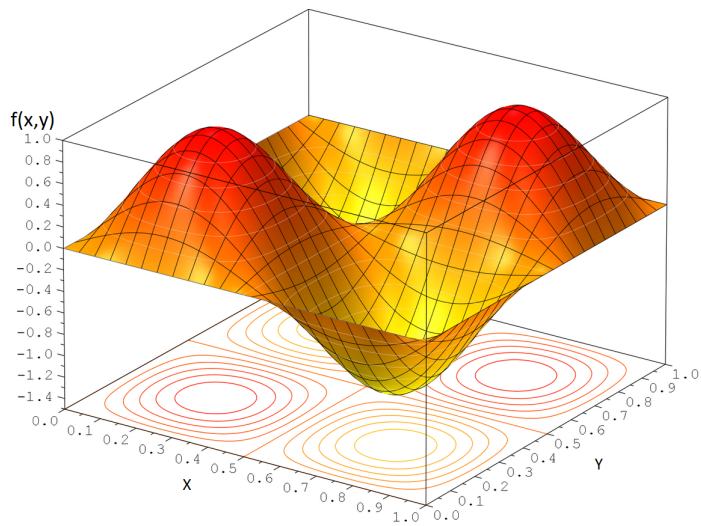
- Las direcciones de descenso se relacionan a las derivadas de la función
- Tenemos:

## Proposición

*Si  $d$  es tal que  $d^T \nabla f(x) < 0$ , entonces  $d$  es una dirección de descenso.*

- Entonces: queremos minimizar y nos dan una dirección de descenso...
- ¿Qué deberíamos hacer?

## Dirección de descenso:



# La idea del Algoritmo de Descenso

**Data:**  $x^0 \in \mathbb{R}^n$ ,  $f$ ,  $k = 0$ .

**while** *Criterio de parada no se satisface* **do**

    Sea  $d^k$  una dirección de descenso en  $x^k$ ;

    Resolver

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k) \quad (\text{etapa de "linesearch"})$$

    Sea  $\lambda^k$  solución óptima de ese problema;

    Sea  $x^{k+1} = x^k + \lambda^k d^k$ ;

$k = k + 1$ ;

**end**

- Hay que especificar todos los detalles...

# La idea del Algoritmo de Descenso

- **La más simple dirección de descenso:**

$$h^k = -\nabla f(x^k)$$

- Da origen al Método del Gradiente o de Cauchy, un método de “primer orden”

- **Y la otra dirección famosa:**

$$h^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$$

- Esta es la Dirección de Newton y da origen al Método de Newton, un método de “segundo orden”.

## ¿De donde viene la Dirección de Newton?

- Supongamos estamos en un punto  $x$ .
- Aproximemos  $f$  mediante su desarrollo de Taylor de orden 2...
- (hagámoslo primero en 1 variable, es decir  $n = 1$ )
- Ahora recordemos nuestro problema “imposible” del comienzo:

$$\text{mín } x^2 + e^{-x}$$

¿Cómo se hace Newton aquí?

## ¿De donde viene la Dirección de Newton?

- Ahora expliquemos Newton en varias variables.
- Primero, recordemos que toda función tiene un Desarrollo de Taylor...
- Definamos la aproximación cuadrática a  $f$  en  $x$ :

$$q(y) = f(x) + \nabla f(x)^T (y - x) + \frac{1}{2} (y - x)^T \nabla^2 f(x) (y - x)$$

- Entonces, aproximamos  $f$  localmente por  $q$ .
- Determinemos el mínimo de esta aproximación cuadrática...

# Método de Newton para minimizar

- Momento: ¿Cómo podemos estar seguros que  $d^k$  sea dirección de descenso?
- La dirección:  $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$ .
- Necesitamos que

$$\nabla f(x^k)^T d^k < 0$$

# Método de Gradiente para minimizar

- **Algoritmo de Gradiente (para minimizar):**

**Data:**  $x^0 \in \mathbb{R}^n$ ,  $f$ ,  $k = 0$ ,  $\epsilon > 0$ .

**while**  $\|\nabla f(x^k)\| > \epsilon$  **do**

    Sea  $d^k = -\nabla f(x^k)$  ;

    Resolver

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k) \quad (\text{etapa de "linesearch"})$$

    Sea  $\lambda^k$  solución óptima de ese problema;

    Sea  $x^{k+1} = x^k + \lambda^k d^k$ ;

$k = k + 1$ ;

**end**

# Método de Newton para minimizar

- **Algoritmo de Newton (para minimizar):**

**Data:**  $x^0 \in \mathbb{R}^n$ ,  $f$ ,  $k = 0$ ,  $\epsilon > 0$ .

**while**  $\|\nabla f(x^k)\| > \epsilon$  **do**

    Sea  $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$  ;

    Resolver

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k) \quad (\text{etapa de "linesearch"})$$

    Sea  $\lambda^k$  solución óptima de ese problema;

    Sea  $x^{k+1} = x^k + \lambda^k d^k$ ;

$k = k + 1$ ;

**end**

## Ejemplo:

- Consideremos la función de  $n$  variables:

$$f(x) = \frac{1}{2}x^T Qx + c^T x + \alpha(5 - x_n)^4$$

donde  $Q \in \mathbb{R}^{n \times n}$  es simétrica y definida positiva,  $c \in \mathbb{R}^n$ ,  $\alpha$  es constante.

- Veamos los métodos de Gradiente y Newton funcionando en un par de casos particulares del problema.
- Mostraremos una implementación computacional en Python.
- Necesitamos el gradiente y hessiano de la función...

- Vemos que, en el ejemplo, ambos llegan a la misma solución pero hay evidentes diferencias de funcionamiento.
- La teoría permite analizar la convergencia de los métodos:
- El Método del Gradiente tiene **convergencia lineal**:
- Si  $x^k$  es la iteración  $k$ -ésima, entonces:

$$\|x^k - x^*\|_2 \leq \alpha \|x^{k-1} - x^*\|_2, k = 1, 2, \dots$$

donde  $0 < \alpha < 1$  y depende de las propiedades de  $f$ .

- El Método de Newton tiene **convergencia cuadrática**:

$$\|x^k - x^*\|_2 \leq \beta \|x^{k-1} - x^*\|_2^2, k = 1, 2, \dots$$

- donde  $\beta$  también depende de las propiedades del problema.

- Gradiente, convergencia lineal:

$$\|x^k - x^*\|_2 \leq \alpha \|x^{k-1} - x^*\|_2$$

- ¿Cuántas iteraciones necesitamos para llegar a una solución  $\epsilon$ -aproximada?

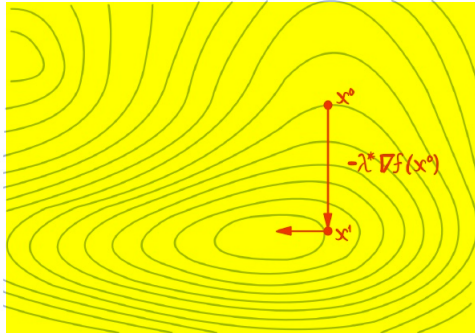
- Newton, convergencia cuadrática:

$$\|x^k - x^*\|_2 \leq \alpha \|x^{k-1} - x^*\|_2^2$$

- ¿Cuántas iteraciones necesitamos para llegar a una solución  $\epsilon$ -aproximada?

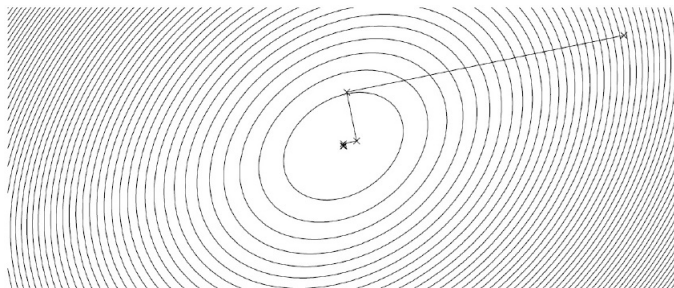
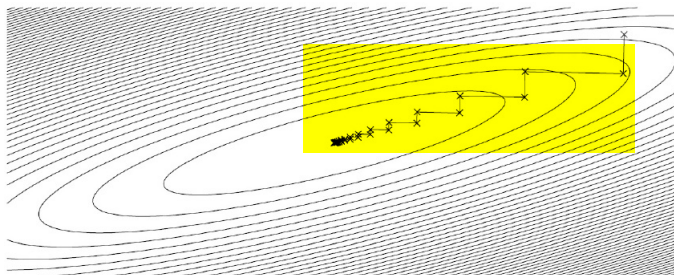
# El problema del Método del Gradiente

- ¿Por qué, a veces, el método del Gradiente tiene convergencia tan lenta? Newton converge mejor que el método del Gradiente?
- El Método del Gradiente se ve muy afectado por las propiedades de curvatura de la función.
- Primero: El avance es siempre “en segmentos en ángulo recto”



# El problema del Método del Gradiente

- Entonces, el avance del método se ve muy afectado por la curvatura de la función:



- (La explicación detallada está en un video complementario)

# La “magia” del Método de Newton

- Pero, entonces, ¿por qué el método de Newton converge mejor que el método del Gradiente?
- Usa más información...
- Notemos que  $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$  es  $-\nabla f(x^k)$  escalado por la inversa del Hessiano...
- La dirección por la que Newton se mueve, “internaliza”, de alguna manera, parte de la información de curvatura de la función.
- Tenemos, entonces, que el Método de Newton tiene convergencia cuadrática, es más rápido en términos del número de iteraciones para llegar a la meta que el Método del Gradiente.
- Pero, ¿qué es lo que realmente nos interesa? ¿Iteraciones o tiempo en llegar?
- ¿cuánto consumimos, de recursos, en cada iteración?

## Sobre la selección del “paso”

- Tanto Gradiente con Newton (y otros métodos) requieren resolver la etapa del “paso de avance” o “linsearch”:

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k)$$

- Definamos  $h(\lambda) = f(x^k + \lambda d^k)$ , el problema  $\min_{\lambda \geq 0} h(\lambda)$  es un problema en 1 variable.
- Este problema ha sido estudiado y se han desarrollado métodos para abordarlo.
- También se ha establecido que sólo hace falta una solución aproximada, y eso se puede lograr con poco esfuerzo.
- (En el video complementario revisaremos algunas ideas).

## Al cierre del tema

- Hemos presentado una visión de algunos de los métodos “clásicos”
- Fueron desarrollados mucho tiempo atrás pero están 100 % vigentes y se usan.
- Hay desarrollos adicionales, para hacerlos menos consumidores de tiempo: Métodos Quasi-Newton (no los estudiaremos).
- Pero la optimización moderna requiere enfoques para nuevos problemas:
- Problemas que surgen en la Internet, Big Data, etc.
- El siguiente tema del curso aborda algunos de estos enfoques.